

NAWIGACJA ROBOTÓW HUMANOIDALNYCH

WYKŁAD 3

Planowanie ruchu i unikanie przeszkód

dr inż. Mateusz Pomianek



Planowanie globalne

Planowanie lokalne

Footstep planning

Kody Python

Plan wykładu

Sekcja 1

Planowanie globalne: przestrzeń stanów, Dijkstra, A*, heurystyki, cost map, wygładzanie

Sekcja 2

Planowanie lokalne: DWA, potential fields, MPC, przeszkody dynamiczne, replanowanie

Sekcja 3

Footstep planning: sekwencja kroków, ograniczenia kinematyczne, schody, trajektoria CoM

Kody Python

A* na grid map, A* z cost map, pole potencjału, ocena trajektorii, generacja kroków

01

SEKCJA

Planowanie globalne

Reprezentacje środowiska, Dijkstra, A, heurystyki i cost map*



Miejsce planowania w architekturze systemu

Wejście: mapa + lokalizacja

Planista dostaje mapę kosztów i bieżącą pozycję robota z estymatora stanu

Wejście: cel misji

Punkt docelowy lub sekwencja celów pośrednich z modułu zarządzania misją

Wyjście: ścieżka globalna

Ciąg punktów w przestrzeni (x,y) lub (x,y,z) przekazywany do planisty lokalnego

Wyjście: cost map

Mapa kosztów przesyłana do lokalnego planisty jako mapa naviagacyjna

Problem planowania ścieżki - formalizacja

- **Przestrzeń stanów:** zbiór wszystkich dopuszczalnych konfiguracji robota $C_{\text{free}} \subseteq C$
 - **Stan początkowy** q_{start} : aktualna pozycja i orientacja robota w mapie
 - **Stan docelowy** q_{goal} : żądana pozycja końcowa misji
- **Ograniczenia:** przeszkody, granice mapy, kinematyka, stabilność robota
- **Funkcja kosztu:** $c(q_{\text{start}}, q_{\text{goal}})$ minimalizowana wzdłuż ścieżki (czas, energia, ryzyko)
- **Ścieżka optymalna:** ciąg $\gamma : [0,1] \rightarrow C_{\text{free}}$ taki, że $\gamma(0)=q_{\text{start}}$, $\gamma(1)=q_{\text{goal}}$
- **Zupełność:** algorytm znajdzie ścieżkę, jeśli istnieje; optymalność: znajdzie najlepszą

Reprezentacje grafowe środowiska

Siatka komórek (Grid)

- Regularna siatka $N \times M$ komórek
- Komórka: wolna / zajęta / koszt
- Łatwa implementacja A*
- Duże mapy = wysoka pamięć
- Rozdzielczość ogranicza precyzję

Graf topologiczny

- Węzły = charakterystyczne miejsca
- Krawędzie = przejazdy między nimi
- Szybkie planowanie wysokopoziomowe
- Niska rozdzielczość geometryczna
- Wymaga mapowania semantycznego

Visibility/Roadmap

- Graf widzialności krawędzi przeszkód
- Trasy wzdłuż wklęsłości przeszkód
- Optymalne w 2D dla znanych map
- Trudne do uogólnienia na 3D
- Rzadko stosowane dla humanoida

Algorytm Dijkstry

Pseudokod Dijkstry

```
1. dist[start] = 0; dist[v] = ∞ dla pozostałych
2. priority_queue Q: (dist, vertex)
3. Q.push((0, start))
4. while Q nie jest pusta:
5.     (d, u) = Q.pop_min()
6.     if d > dist[u]: continue
7.     for (v, w) in sąsiedzi(u):
8.         nowy_koszt = dist[u] + w
9.         if nowy_koszt < dist[v]:
10.            dist[v] = nowy_koszt
11.            poprzednik[v] = u
12.            Q.push((nowy_koszt, v))
13. odtwórz ścieżkę przez poprzednik[]
```

Właściwości

- Gwarantuje optymalność
- Brak heurystyki – eksploruje we wszystkich kierunkach
- Złożoność: $O((V+E) \log V)$
- Wolny dla dużych map
- Dobry jako punkt odniesienia
- Podstawa A* i Bellmana-Forda

Algorytm A* - planowanie z heurystyką

Pseudokod A*

$f(n) = g(n) + h(n)$

$g(n)$ - koszt dotarcia od startu do n

$h(n)$ - heurystyka: szacunek do celu

```
1. open_set = {start}
2. g[start] = 0; f[start] = h(start)
3. while open_set nie jest pusta:
4.     u = węzeł z min f w open_set
5.     if u == goal: odtwórz ścieżkę
6.     open_set.remove(u)
7.     for v in sąsiedzi(u):
8.         tentative_g = g[u] + cost(u,v)
9.         if tentative_g < g[v]:
10.            g[v] = tentative_g
11.            f[v] = g[v] + h(v)
12.            parent[v] = u
```

Właściwości

- Optymalny gdy h admissible
- Szybszy od Dijkstry dzięki h
- h euklidesowa: $\sqrt{dx^2+dy^2}$
- h Manhattan: $|dx|+|dy|$
- Im lepsza h , tym mniej węzłów
- Złożoność: $O(b^d)$, b =branching

A* na mapie zajętości (grid map)

```
import heapq, numpy as np

def astar(grid, start, goal):
    """A* na siatce 2D. grid[r][c]=0: wolne, 1: zajete."""
    rows, cols = grid.shape
    h = lambda r, c: abs(r-goal[0]) + abs(c-goal[1])
    g = {start: 0}
    parent = {start: None}
    pq = [(h(*start), start)]
    while pq:
        _, u = heapq.heappop(pq)
        if u == goal:
            path = []
            while u:
                path.append(u)
                u = parent[u]
            return path[::-1]
        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            v = (u[0]+dr, u[1]+dc)
            if 0<=v[0]<rows and 0<=v[1]<cols and grid[v]==0:
                ng = g[u] + 1
                if ng < g.get(v, 1e9):
                    g[v]=ng; parent[v]=u
                    heapq.heappush(pq, (ng+h(*v), v))
```

Opis: Manhattan h – dopuszczalna i spójna; 4-kierunkowa sąsiedztwo; zwraca listę komórek ścieżki

Heurystyki w A* - właściwości i zastosowanie

Heurystyka Manhattan (L1)

- $h(n) = |dx| + |dy|$
- Idealna dla siatek z ruchem 4-kierunkowym
- Dopuszczalna: nigdy nie przeszacowuje kosztu
- Bardzo szybka – prosta operacja arytmetyczna
- Dla ruchu 8-kierunkowego: lekko nieprecyzyjna

Heurystyka euklidesowa (L2)

- $h(n) = \sqrt{dx^2 + dy^2}$
- Odpowiednia dla ciągłego ruchu w 2D/3D
- Dopuszczalna dla humanoida (metryczny koszt)
- Droższa obliczeniowo (sqrt), lecz precyzyjniejsza
- Zalecana gdy ścieżka jest wygładzana

Planowanie z uwzględnieniem kosztów terenu

- **Cost map** zastępuje binarną mapę wolne/zajęte gradientem kosztów
- **Cel**: preferowanie bezpiecznych, szerokich przejść zamiast najkrótszej trasy geometrycznej
- **Warstwy kosztu**: inflacja wokół przeszkód, nachylenie terenu, wskaźnik stabilności, gęstość ruchu
- **Implementacja**: komórki siatki mają wartość $c \in [0, 255]$; A* używa $\text{cost}(u,v) = 1 + c(v)/50$
- **Efekt**: trasa omija obrzeża mebli i wąskie przejścia nawet gdy są geometrycznie wolne
- **Dynamiczna aktualizacja**: nowe obserwacje LiDAR/kamery zmieniają cost map online
- **Parametryzacja**: wagi warstw dobierane eksperymentalnie lub przez uczenie maszynowe

A* z mapą kosztów terenu

```
import heapq, numpy as np

def astar_cost(grid, cost_map, start, goal):
    """A* z kosztem komórki z cost_map [0-100]."""
    rows, cols = grid.shape
    h = lambda r, c: np.hypot(r-goal[0], c-goal[1]) # euklidesowa
    g = {start: 0.0}
    parent = {start: None}
    pq = [(h(*start), start)]
    while pq:
        _, u = heapq.heappop(pq)
        if u == goal:
            path, node = [], u
            while node: path.append(node); node = parent[node]
            return path[::-1]
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(1,1),(-1,1),(1,-1)]:
            v = (u[0]+dr, u[1]+dc)
            if 0<=v[0]<rows and 0<=v[1]<cols and grid[v]==0:
                step = np.hypot(dr,dc) * (1 + cost_map[v]/50.0)
                if (ng := g[u]+step) < g.get(v,1e9):
                    g[v]=ng; parent[v]=u
                    heapq.heappush(pq,(ng+h(*v),v))
```

Opis: 8-kierunkowe sąsiedztwo; koszt przejścia = odległość * (1 + wartość_kosztu/50); ruch po skosie droższy

Wygładzanie ścieżki globalnej

- A* na siatce zwraca ścieżkę z ostrymi załamaniami przy każdej zmianie kierunku
- Wygładzanie: konieczne przed przekazaniem ścieżki do generatora kroków lub MPC
- Metoda 1 – interpolacja B-spline: krzywa przechodzi blisko punktów kontrolnych, zachowuje ciągłość krzywizny
- Metoda 2 – skrótów (shortcutting): iteracyjne usuwanie pośrednich punktów, jeśli prosta jest wolna od przeszkód
- Metoda 3 – filtr Gaussa 1D: ważona średnia po sąsiednich punktach wzdłuż ścieżki
- Metryka: kąt ugięcia $< 15^\circ$ między kolejnymi odcinkami jako warunek akceptacji
- Ważne: wygładzona ścieżka musi pozostawać w wolnej przestrzeni – weryfikacja kolizji po wygładzeniu

02

SEKCJA

Planowanie lokalne i unikanie przeszkód

DWA, potential fields, MPC, przeszkody dynamiczne



Po co potrzebny planista lokalny?

Świat dynamiczny

Mapa globalna nie uwzględnia ludzi, wózków, otwieranych drzwi – planista lokalny reaguje na bieżąco

Niedoskonała mapa

Sensory mają szum, mapa jest zbudowana ze starego skanu – lokalna mapa jest zawsze świeższa

Ograniczenia dynamiki

Robot nie może skręcić dowolnie ostro ani zatrzymać się natychmiast – kinematyka musi być uwzględniona

Horyzont predykcji

Decyzje na 1–3 s do przodu wystarczają; pełny replan jest zbyt kosztowny w każdym cyklu (10–50 Hz)

Pola potencjału (Potential Fields)

Wzory sił

$U_{\text{attr}}(q) = \frac{1}{2} \cdot k_{\text{att}} \cdot d^2(q, q_{\text{goal}})$
– atrakcja do celu; rośnie z odległością

$U_{\text{rep}}(q) = \frac{1}{2} \cdot k_{\text{rep}} \cdot (1/d - 1/d_0)^2$
gdy $d < d_0$, inaczej 0
– odpychanie od przeszkód
 d_0 – promień wpływu przeszkody

$F(q) = -\nabla U(q)$
 $= F_{\text{attr}} + F_{\text{rep}}$

Ruch: $q_{\text{new}} = q + \alpha \cdot F(q)$
 α – krok iteracji

Właściwości

- Prosta implementacja
- Działa w czasie rzeczywistym
- Problem: minima lokalne
- Oscylacje w wąskich przejściach
- Trudna parametryzacja k_{att} , k_{rep}
- Rozwiązanie: potencjały harmoniczne lub RRT

Dynamic Window Approach (DWA) - idea

- **Idea:** w każdym kroku czasowym szukaj najlepszej pary prędkości (v, ω) z okna osiągalnego
- **Okno dynamiczne:** $\{(v, \omega) \mid v \in [v-a_{\max} \cdot dt, v+a_{\max} \cdot dt], \omega \text{ analogicznie}\}$
- **Ocena trajektorii:** dla każdego kandydata (v, ω) symuluj ruch przez T sekund (np. $T=2$ s)
- **Funkcja celu:** $G(v, \omega) = \alpha \cdot \text{heading} + \beta \cdot \text{clearance} + \gamma \cdot \text{velocity}$
- **heading:** wyrównanie kierunku ruchu z celem globalnym
- **clearance:** odległość od najbliższej przeszkody wzdłuż trajektorii
- **velocity:** nagroda za wyższą prędkość do przodu
- **Wybierz** $(v^*, \omega^*) = \operatorname{argmax} G$ – wykonaj jeden krok, przelicz w kolejnym cyklu

Model Predictive Control - intuicja

- **MPC** optymalizuje sekwencję sterowań $u_0, u_1, \dots, u_{N-1}$ na horyzoncie N kroków
- **Model predykcji**: $x_{t+1} = f(x_t, u_t)$ – uproszczony model kinematyczny/dynamiczny robota
- **Funkcja celu**: minimalizacja odchylenia od ścieżki + kosztu sterowania + penalizacja za przeszkody
- **Ograniczenia**: $|v| \leq v_{\max}, |\omega| \leq \omega_{\max}$, brak kolizji, stabilność ZMP
- **Receding horizon**: po wyznaczeniu sekwencji wykonuje się tylko u_0 , a plan jest aktualizowany
- **Zalety**: jawne ograniczenia dynamiczne, optymalne zarządzanie kolizjami w horyzoncie
- **Wada**: wysoki koszt obliczeniowy (solver QP/NLP); wymaga dedykowanego CPU lub GPU

Pole potencjału - lokalny wektor ruchu

```
import numpy as np

def potential_field_move(robot_pos, goal_pos, obstacles,
                        k_att=1.0, k_rep=100.0, d0=1.5):
    """Wyznacza wektor ruchu z pol potencjalnych."""
    robot = np.array(robot_pos, dtype=float)
    goal = np.array(goal_pos, dtype=float)
    # Sila atrakcji do celu
    f_att = k_att * (goal - robot)
    # Suma sil odpychania od przeszkod
    f_rep = np.zeros(2)
    for obs in obstacles:
        diff = robot - np.array(obs)
        d = np.linalg.norm(diff) + 1e-6
        if d < d0:
            f_rep += k_rep * (1/d - 1/d0) * diff / d**3
    total = f_att + f_rep
    return total / (np.linalg.norm(total) + 1e-6) # wektor jednostkowy
```

Opis: Zwraca znormalizowany wektor ruchu; mnożymy przez Δt i prędkość robota, by uzyskać przemieszczenie

Ocena kandydatów trajektorii (uproszczone DWA)

```
import numpy as np

def score_trajectory(v, omega, robot_state, goal, obstacles,
                    T=2.0, dt=0.1,
                    alpha=0.4, beta=0.4, gamma=0.2):
    """Ocenia trajektorie (v, omega) w horyzoncie T sekund."""
    x, y, yaw = robot_state
    min_clearance = 1e9
    for _ in range(int(T / dt)):
        x += v * np.cos(yaw) * dt
        y += v * np.sin(yaw) * dt
        yaw += omega * dt
        for obs in obstacles:
            min_clearance = min(min_clearance, np.hypot(x-obs[0], y-obs[1]))
    heading = -abs(np.arctan2(goal[1]-y, goal[0]-x) - yaw)
    clearance = min_clearance
    return alpha*heading + beta*clearance + gamma*v
```

Opis: Wyższy score = lepsza trajektoria; wywoływane dla N par (v, ω) , wybierana para z max score

Unikanie ludzi i przeszkód dynamicznych

Detekcja i predykcja

- **Detekcja ludzi:** YOLO, PointNet na chmurze LiDAR
- **Śledzenie** (tracking): filtr Kalmana per obiekt
- **Predykcja:** stałe velocity, Social Force Model
- **Horyzont predykcji:** 1–3 s wystarczy
- **Niepewność:** elipsa kowariancji wokół ścieżki

Planowanie z uwzględnieniem ruchu

- **Strefa bezpieczeństwa:** okrąg lub elipsa wokół człowieka
- **Prędkość** przeszkody skaluje rozmiar strefy
- **ORCA** (Optimal Reciprocal Collision Avoidance)
- **Social navigation:** min. 1,2 m od człowieka
- Zatrzymanie gdy człowiek zbliży się $< 0,5$ m

Replanowanie online - kiedy i jak

Replan globalny (rzadki)

- Gdy cel misji zostaje zmieniony
- Gdy mapa zasadniczo się zmienia (*zamknięte przejście*)
- Co 5–30 sekund lub na żądanie operatora
- Kosztowny obliczeniowo – całkowite A^*
- Alternatywa: D^* Lite (*przyrostowy A^**)

Replan lokalny (częsty)

- Co 50–100 ms w każdym cyklu planisty
- Okno lokalne wokół bieżącej pozycji robota
- Reaguje na nowe przeszkody dynamiczne
- Możliwy na CPU embarked 1–2 ms
- Brak zmiany ścieżki globalnej

Konflikt między planem a stabilnością

- **Problem:** optymalny manewr unikania może wymagać szybkiego skrętu lub dużej prędkości bocznej
- **Konsekwencja:** środek masy wychodzi poza wielobok podparcia → utrata równowagi
- **Reguła nadrzędna:** stabilność > unikanie kolizji > optymalizacja trasy
- **Implementacja:** każda kandydatka trajektoria lokalnej jest filtrowana przez weryfikator ZMP
- **Weryfikator ZMP:** odrzuca trajektorie, gdzie przewidywany ZMP wychodzi poza margines bezpieczeństwa
- **Efekt:** planista lokalny preferuje wolniejsze, bardziej konserwatywne manewry w trudnym terenie
- **Parametr:** `safety_margin_zmp` = 2–4 cm; dla schodów i pochyłości zwiększony do 5–8 cm

03

SEKCJA

Planowanie kroków i trajektorii humanoida

*Footstep planning, ograniczenia, schody, trajektoria
CoM*



Od ścieżki geometrycznej do sekwencji kroków

1

Ścieżka bazy

Globalna trasa (x,y) lub (x,y,z) wyznaczona przez A^* ; reprezentuje ruch CoM lub miednicy

2

Wzorzec chodu

Parametry: długość kroku l , szerokość w , czas fazy T_{step} , T_{swing} ; regularne lub adaptacyjne

3

Wyznaczenie stóp

Dla każdego punktu ścieżki wyznacz cele stopy: offset lewo/prawo $\pm w/2$, orientacja = yaw trasy

4

Weryfikacja

Sprawdź długość kroku, osiągalność kinematyczną, kontakt z terenem, margines ZMP i stabilność

Footstep planning - parametry i ograniczenia

- **Długość kroku:** typowo 0,2–0,5 m; zależy od prędkości i stabilności (przy niskiej prędkości krócej)
- **Szerokość kroku:** 0,1–0,25 m; za mała = utrata stabilności, za duża = kolizja nóg
- **Orientacja stopy:** dostosowana do krzywizny ścieżki i pochylenia terenu (normal surface)
- **Strefy dozwolonego kontaktu** (foothold regions): płaskie, niegrzybowe obszary $\geq$ stopa robota
- **Kolejność kroków:** lewa–prawa naprzemiennie; podwójne podparcie $\geq$ 20% cyklu
- **Wykluczenie:** krawędzie, śliskie powierzchnie, zbyt strome pochylenie ($>20^\circ$), wąskie progi
- **Planowanie w 3D:** wymagane gdy teren ma zmienną wysokość (schody, kamienie, nierówności)

Ograniczenia kinematyczne i stabilnościowe

Ograniczenia kinematyczne

- **Zasięg nogi:** max odległość stopa–biodro $\leq l_{\max}$
- **Min. odstęp:** stopy nie mogą nachodzić na siebie
- **Zakres stawów:** każdy staw w przedziale $[q_{\min}, q_{\max}]$
- **Prędkość stawów:** $\omega_{\text{joint}} \leq \omega_{\max}$ (przepustowość aktuatora)
- **Konfiguracja singularna:** unikanie pełnego wyprostu kolana

Ograniczenia stabilnościowe

- **ZMP** wewnątrz wieloboku podparcia z marginesem
- **Capture Point** w dopuszczalnej strefie landing
- **Odchylenie tułowia** $\leq \pm 15^\circ$ od pionu
- **Prędkość CoM** ograniczona przez marginesy LIPM
- **Przy schodach:** dodatkowy margines boczny +3 cm

Planowanie na schodach i nierównościach

- **Detekcja schodów:** segmentacja chmury punktów; klasyfikacja płaskich poziomych płaszczyzn
- **Wysokość stopnia:** robot musi unieść stopę o ($h_{\text{step}} + \text{clearance}$) ponad krawędź
- **Miejsca kontaktu:** wyznaczane na środku głębokości stopnia z marginesem ≥ 2 cm od krawędzi
- **Kolejność działań:** wejście przodem – lewa noga wyżej $\rightarrow$ prawa noga dołącza, powtórz
- **Trajektoria stopy w powietrzu:** krzywa beziera nad stopniem; max wysokość = $h_{\text{step}} + 5$ cm
- **Kontrola ciała:** tułów pochyla się nieznacznie do przodu przy wchodzeniu, do tyłu przy schodzeniu
- **Zabezpieczenie:** jeśli wysokość stopnia > 25 cm lub brak miejsca kontaktu – zatrzymaj misję

Generacja sekwencji kroków (footstep planner)

```
import numpy as np

def generate_footsteps(path_xy, step_length=0.35,
                      step_width=0.18) -> list:
    """Generuje naprzemienne kroki L/P wzdłuż ścieżki."""
    footsteps = []
    left_foot = True # zaczyna lewa noga
    dist_accum = 0.0
    for i in range(1, len(path_xy)):
        dx = path_xy[i][0] - path_xy[i-1][0]
        dy = path_xy[i][1] - path_xy[i-1][1]
        dist_accum += np.hypot(dx, dy)
        if dist_accum >= step_length:
            yaw = np.arctan2(dy, dx)
            sign = 1.0 if left_foot else -1.0
            foot_x = path_xy[i][0] - np.sin(yaw)*sign*step_width/2
            foot_y = path_xy[i][1] + np.cos(yaw)*sign*step_width/2
            footsteps.append({'x':foot_x,'y':foot_y,'yaw':yaw,'left':left_foot})
            left_foot = not left_foot
            dist_accum = 0.0
    return footsteps
```

Opis: Offset $\pm \text{step_width}/2$ prostopadłe do kierunku ruchu; 'left' flaga steruje naprzemiennością L/P.

Weryfikacja wykonalności kroku

```
import numpy as np

def generate_footsteps(path_xy, step_length=0.35,
                      step_width=0.18) -> list:
    """Generuje naprzemienne kroki L/P wzdłuż ścieżki."""
    footsteps = []
    left_foot = True # zaczyna lewa noga
    dist_accum = 0.0
    for i in range(1, len(path_xy)):
        dx = path_xy[i][0] - path_xy[i-1][0]
        dy = path_xy[i][1] - path_xy[i-1][1]
        dist_accum += np.hypot(dx, dy)
        if dist_accum >= step_length:
            yaw = np.arctan2(dy, dx)
            sign = 1.0 if left_foot else -1.0
            foot_x = path_xy[i][0] - np.sin(yaw)*sign*step_width/2
            foot_y = path_xy[i][1] + np.cos(yaw)*sign*step_width/2
            footsteps.append({'x':foot_x,'y':foot_y,'yaw':yaw,'left':left_foot})
            left_foot = not left_foot
            dist_accum = 0.0
    return footsteps
```

Opis: Trzy sprawdzenia: długość wzdłużna, odchylenie boczne i skręt; zwraca dict z diagnostyką

Trajektoria środka masy a sekwencja kroków

- **CoM** (Centre of Mass) musi śledzić zaplanowane kroki przez model LIPM (Linear Inverted Pendulum)
- **Przy podwójnym podparciu:** CoM przesuwa się od poprzedniej stopy do następnej
- **Przy pojedynczym podparciu** (faza swing): CoM podąża parabolą wyznaczoną przez ZMP
- **Sprzężenie:** sekwencja kroków $\rightarrow$ trajektoria ZMP $\rightarrow$ trajektoria CoM (generator QP)
- **Weryfikacja online:** $ZMP(t) \in \text{convex hull}(\text{stopa_oparta})$ przez cały cykl kroku
- **Narzędzie:** biblioteki Choreonoid, OpenHRP, ROS2 Whole Body Controller
- **Kompromis:** krótsze, częstsze kroki = lepsza stabilność CoM kosztem prędkości ruchu

Dobór planisty do konkretnej aplikacji

Laboratorium badawcze

A* + DWA; prosta cost map; footstep planer 2D; iteracyjna weryfikacja ZMP; MuJoCo do testów

Robot usługowy (biuro)

A* z cost map + Social DWA; detekcja YOLO; adaptacyjne kroki; zatrzymanie przy ludziach < 1 m

Ratownictwo (gruz)

3D A* lub RRT*; footstep w 3D z height map; MPC lokalny; konserwatywne marginesy ZMP

Środowisko zatłoczone

ORCA lub S-MPC; krótki horyzont predykcji (0,5 s); priority queue stref bezpiecznych; niskie prędkości

Schody i poziomy

Hierarchiczny planer: globalny graf pięter + lokalny 3D footstep; detekcja stopni z point cloud

Sport/zawody

MPC z agresywnym horyzontem; dynamiczny chód biegowy; uproszczony footstep dla szybkości kosztem marginesu ZMP

PODSUMOWANIE

01

Planowanie globalne (A^* , Dijkstra) wyznacza optymalną trasę; heurystyki i cost map sterują wyborem ścieżki.

02

Planista lokalny (DWA, MPC, potential fields) reaguje na zmiany środowiska i przeszkody dynamiczne.

03

Stabilność ma wyższy priorytet niż optymalność trasy – planista musi uwzględniać margines równowagi.

04

Footstep planning tłumaczy geometryczną ścieżkę na fizyczną sekwencję kroków z weryfikacją kinematyczną.

05

Trzy poziomy decyzji – globalny, lokalny, footstep – są sprzężone i muszą działać w spójnej pętli.

Zadanie projektowe i pytania problemowe

- Dlaczego heurystyka Manhattan nie jest dopuszczalna dla siatek z ruchem 8-kierunkowym?
- Co się dzieje z polem potencjału, gdy robot wpadnie w lokalne minimum? Zaproponuj wyjście.
- Jaką wartość parametru α i β wybrałbyś dla DWA w zatłoczonym korytarzu? Uzasadnij.
- **Zadanie projektowe:** zaprojektuj planistę kroków dla humanoida poruszającego się w korytarzu 1,5 m szerokości z napotykanymi ludźmi. Opisz: jakie dane wejściowe są potrzebne, jak wyznaczasz miejsca kontaktu stóp, jak weryfikujesz każdy krok, jak reaguje planista gdy droga jest zablokowana przez >3 s.
- **Bonus:** zaimplementuj wygładzanie B-spline dla ścieżki zwróconej przez A* i porównaj liczbę kroków koniecznych do pokonania tej samej trasy przed i po wygładzeniu.